University of Applied Sciences and Arts
Northwestern Switzerland
School of Computer Science
Institute for Mobile and Distributed Systems (IMVS)

Master of Science in Engineering (MSE)

Project P8

Enhancing Web Accessibility Standards

Implementation of User-Centric Customization Interfaces and CSS-based Contrast Optimization

---

Author: **Florian Schnidrig**

Advisor: Prof. Dierk König

Profile: Computer Science

Date: Summer 2026 (TBD)

Contents

1. Abstract 1
2. Acknowledgement 1
3. Introduction 2
   3.1. Objective of this Project 2
4. Theory 3
   4.1. CSS Media Queries 3
      4.1.1. Reduced Motion 4
      4.1.2. Contrast 4
      4.1.3. Forced Colors 4
      4.1.4. Reduced Transparency 5
      4.1.5. Color Scheme 5
      4.1.6. Inverted Colors 5
      4.1.7. Accessing Media Features through JavaScript 5
      4.1.8. Changes in the World of Media Queries 6
      4.1.9. Deprecated Features 6
      4.1.10. Security Concerns 6
   4.2. CSS Custom Properties 7
      4.2.1. Global State Management 8
      4.2.2. Custom Property Toggle 10
   4.3. CSS Functions 12
      4.3.1. CSS Custom Functions 12
   4.4. View Transition API 14
   4.5. Color Spaces 16
      4.5.1. RGB 16
      4.5.2. HSL 16
      4.5.3. CIELAB Colors 17
      4.5.4. HWB 17
   4.6. Contrast Perception 19
      4.6.1. Relative Luminance 19
      4.6.2. Perceptual Color Models 20
   4.7. Canvas 23
   4.8. Ishihara 24
      4.8.1. Functional Application in Contrast Testing 24
5. Methodology 25
   5.1. Research and Ideation 25
   5.2. Prototyping 25
   5.3. Mathematical Foundation 26
   5.4. Validation and Verification 26
6. Implementation 27
   6.1. Preferences Widget 27
      6.1.1. JavaScript Part 28

6.1.2. CSS Part 31
6.1.3. The Paradox of Reduced Motion Transitions 32
6.1.3.1. Animating Properties 33
6.1.3.2. View Transition 33
6.2. Contrast Color Function 35
6.3. Interactive Ishihara Plate 38
6.4. (Accessible Drag and Drop Suggestion) 40
7. Discussion 44
8. Conclusions 46
8.1. Limitations 46
8.2. Future Work 46
Bibliography 48
List of Figures 51
List of Listings 52
List of Aids 54
Artificial Intelligence and Language Models 54
MSE-Declaration of Originality 55

1. Abstract

This project explores native CSS and JavaScript features related to accessibility concerns, discusses the theory behind contrast perception in WCAG 2.x, and introduces the new perceptual approach proposed in the current WCAG 3.0 working draft. In addition, it examines patterns and techniques such as global state management with CSS custom properties, CSS attribute selectors, and experimental at-rules that can be used to create more individualized and accessible user experiences.

The accompanying repository contains interactive example pages and code snippets for all implementations discussed in this documentation and additional experiments. These examples allow the presented functionality to be explored in a practical and interactive way. The repository also includes this documentation both as Typst source code and as a rendered PDF: https://github.com/Flaverus/P8_sandbox/tree/main

The three main tools developed during this project are:

- A user-centric Preferences Widget that enables persistent website-specific accessibility settings beyond native operating system restrictions.
- A Custom Contrast Color Function based on the OkLCH color space that softens harsh contrast through perceptual color mixing.
- An Interactive Ishihara Plate generator designed as a developer tool for evaluating color combinations and simulating color vision deficiencies such as `Protanopia`, `Deuteranopia`, and `Tritanopia`.

This project demonstrates that modern web standards already provide the foundation for highly personalized and perceptually aware accessibility solutions without requiring large external frameworks or overly complex configurations.

2. Acknowledgement

[TBD]

3. Introduction

The previous project, *Accessibility on the Web – Where we Stand* [1], examined the fundamental principles of digital inclusion in modern web applications. It explored how semantic HTML forms the foundation of accessibility, provided an overview of both common and lesser-known native elements, and clarified the roles of ARIA and WCAG. In addition, it introduced the basics of accessibility debugging, testing, and the practical use of screen readers.

Building on that foundation, this project focuses on essential aspects of accessibility that may appear less technical in the traditional sense, but are equally important for creating an inclusive digital world. While the previous project concentrated primarily on the structural and technical mechanisms behind the scenes, it did not examine the visual design of web applications. These design decisions are far more than aesthetic choices. They play a decisive role in determining whether a web application becomes a welcoming space or a digital dead end for many users.

3.1. Objective of this Project

This project aims to explore accessibility in greater depth by examining ways to build on and extend existing browser standards. It also investigates additional techniques and approaches that can simplify the user journeys of people with impairments, making web applications more user-friendly and adaptable to individual needs.

4. Theory

The following chapter introduces CSS and JavaScript features, along with patterns and standards that can serve as a foundation for improving visual accessibility. These techniques help create a better user experience, particularly for people with visual impairments.

4.1. CSS Media Queries

CSS Media Queries allow developers to apply different styles based on the characteristics of the device or environment used to display a web page. This mechanism is commonly used to create responsive layouts that adapt their appearance according to the available screen width.

Listing 1 below illustrates the syntax of a media query as defined by the W3C.

```
@media <media-query-list> {
  <rule-list>
}
```

Listing 1: Syntax of a media query, where `<media-query-list>` *contains* `<media-type>` *values,* `<media-feature>` *values, and logical operators.*

In web development, the `screen` media type is the most commonly used, but media queries can also target printers by using `print`. If no media type is specified, the default value is `all`, which applies the styles to all devices.

Media features are where media queries become particularly powerful, as more than 40 features are currently available. They allow styles to respond to specific characteristics of the user's environment. Common examples include `max-width` and `max-height`, as well as their counterparts `min-width` and `min-height`. These features are widely used to create responsive websites with layouts optimized for different screen sizes.

Listing 2 demonstrates a media query that uses the `screen` media type together with the `max-width` media feature.

Some media features take accessibility needs and user preferences into account. They make it possible to adapt the presentation of a website to individual requirements. Despite their importance, they are still used less frequently than they should be. The following media features are particularly relevant when building web applications that aim to be accessible to all users. [2]

```css
@media screen and (max-width: 768px) {
  body {
    background-color: lightskyblue;
  }
}
```

Listing 2: Example of a media query that changes the background color on smaller screens using the `screen` media type and the `max-width` media feature.

4.1.1. Reduced Motion

The `prefers-reduced-motion` media feature indicates whether a user has enabled a system setting that reduces non-essential motion. If the value is set to `reduce`, animations and transitions should be limited to the bare minimum.

A value of `no-preference` indicates that the user has not expressed a preference for reduced motion, allowing animations and transitions to be displayed without restriction. [3]

4.1.2. Contrast

The `prefers-contrast` media feature allows users to indicate whether they prefer content to be presented with higher or lower contrast. A value of `no-preference` indicates that the user has not configured a contrast preference.

If the value is set to `more`, the user requests a higher level of contrast. Conversely, a value of `less` indicates that a lower contrast is preferred.

The value `custom` indicates that the user prefers a specific set of colors. This color palette is typically defined through the `forced-colors` media feature, which is explained in the following section. [4]

4.1.3. Forced Colors

The `forced-colors` media feature detects whether the user agent has enabled a forced color mode, such as Windows High Contrast. This media feature has the value `active` when such a mode is enabled and `none` when it is not.

When forced colors mode is active, many color values defined in the style sheet are overridden by the browser during the painting process. The exact colors applied depend on the semantic role of each element. In addition, browsers may draw backplates behind text to preserve legibility and maintain sufficient contrast when text appears on top of images or patterned backgrounds.

Developers are generally not encouraged to create a separate design specifically for users who enable forced colors mode. Instead, this media query should be used to make targeted adjustments when certain components do not work well in this context. For example, a button that relies solely on `box-shadow` to stand out from

the background may lose its visual distinction. In such cases, replacing the shadow with a visible border can provide a more robust solution. [5]

4.1.4. Reduced Transparency

The `prefers-reduced-transparency` media feature indicates whether a user has enabled a system setting to reduce transparent or translucent visual effects. If the value is set to `reduce`, transparency effects such as blurred overlays or frosted glass backgrounds should be minimized or removed. A value of `no-preference` indicates that the user has not expressed a preference and that the default design can be used.

Reducing transparency can improve contrast and readability, particularly for users who find translucent interface elements distracting or difficult to perceive.

Support for this media feature is currently limited and it is not yet fully supported by Firefox and Safari. [6]

4.1.5. Color Scheme

A more widely adopted media feature is `prefers-color-scheme`. It indicates whether the user prefers a light or dark color theme configured through the operating system or the user agent. The value `light` represents a light color theme, while `dark` represents a dark color theme. [7]

4.1.6. Inverted Colors

The `inverted-colors` media feature detects whether the user agent is displaying content with inverted colors. Color inversion can introduce visual side effects, such as shadows appearing as highlights, which may reduce the readability of the content. By responding to this media feature, developers can make targeted adjustments to preserve the visual integrity of the page.

Support for this media feature is currently limited and, at the time of writing, it is only supported by Safari. [8]

4.1.7. Accessing Media Features through JavaScript

The `window` interface provides the `matchMedia()` method, which returns a `MediaQueryList` object. This object can be used to determine whether the current `document` matches a given media query string. Listing 3 demonstrates this approach using the `prefers-color-scheme` media feature.

```
const isDarkTheme = window.matchMedia('(prefers-color-scheme:
dark)').matches;
```

Listing 3: Checking whether the user's system settings prefer a dark color scheme.

This allows developers to respond to user preferences not only in CSS, but also within JavaScript logic and application-specific features. [9]

4.1.8. Changes in the World of Media Queries

CSS media queries continue to evolve, but one important aspect of visual accessibility is still not addressed. Different forms of color blindness and other impairments that affect color perception are not yet covered by an official media feature.

An open issue in the W3C's `csswg-drafts` repository proposes a new media feature called `color-vision-adjustment`. Its goal is to improve web accessibility by allowing developers to respond directly to the needs of users with color vision deficiencies.

The proposal includes support for several specific conditions, including protanopia (red deficiency), deuteranopia (red-green deficiency), tritanopia (blue-yellow deficiency), and achromatopsia (near-total color blindness, resulting in grayscale vision). These impairments were previously investigated in the P7 project through browser bookmarklets designed to simulate the corresponding visual deficiencies.

Further details regarding the implementation and theoretical background of these simulations can be found in the P7 project documentation: https://accessible-web-initiative.gitbook.io/accessibility-on-the-web-where-we-stand.

A media feature of this kind would increase awareness of color vision deficiencies and provide developers with a standardized way to adapt interfaces to the needs of affected users. [10]

4.1.9. Deprecated Features

Earlier versions of CSS included media types such as `embossed`, `aural`, and `braille`, which were intended to address specific accessibility needs. These media types were later deprecated because the distinction between device categories was expected to become less meaningful over time. In addition, media types describe classes of devices rather than the actual capabilities they support. [11], [12]

4.1.10. Security Concerns

Information exposed through media queries can be used to build a browser fingerprint, which may allow a device to be identified and tracked across websites. To reduce this risk, browsers may deliberately modify or generalize certain reported values.

Some browsers also provide settings that further limit this information. For example, Firefox offers the "Resist Fingerprinting" option, which causes many media queries to return standardized values instead of device-specific settings. [12]

4.2. CSS Custom Properties

CSS custom properties, also known as CSS variables, are named values that can be reused throughout a style sheet. Their value depends on the scope in which they are defined, but within that scope they evaluate to the same value wherever they are used. This makes styles easier to maintain and reduces duplication.

Custom properties are defined using a name that begins with two dashes, `--`, followed by a descriptive identifier. Their value can be accessed using the CSS `var()` function. Listing 4 demonstrates how the custom property `--primary-color` can be defined once and reused throughout a website. This approach is particularly useful when implementing multiple themes, such as light and dark mode, because colors can be changed in a single central location. [13]

```css
:root {
  --primary-color:          #204ccf;
  --primary-color-contrast: #ffffff;
}

button.primary {
  background-color: var(--primary-color);
  color: var(--primary-color-contrast);
}

h1, h2, h3 {
  color: var(--primary-color);
}
```

Listing 4: Defining a primary color custom property that can be reused throughout the website.

The `@property` at-rule allows custom properties to be defined more precisely. It makes it possible to specify the expected value type using `syntax`, whether the property is inherited by default using `inherits`, and an initial value using `initial-value`. Possible syntax definitions include `<color>`, `<number>`, and `<url>`. Listing 5 shows how the previous `--primary-color` example can be expressed using this at-rule. [14], [15]

```
@property --primary-color {
  syntax: "<color>";
  inherits: false;
  initial-value: #204ccf;
}
```

Listing 5: Defining a primary color custom property using the `@property` at-rule.

As shown in Listing 6, custom properties can also be defined in JavaScript using `registerProperty()` or using `setProperty()`. [16], [17]

```
window.CSS.registerProperty({
  name: '--primary-color',
  syntax: '<color>',
  inherits: false,
  initialValue: '#204ccf ',
});

const root = document.documentElement;
root.style.setProperty('--primary-color-contrast', '#ffffff');
```

Listing 6: Using `registerProperty()` and `setProperty()` in JavaScript to define CSS custom properties.

Custom properties can also be read in JavaScript using `getPropertyValue()`, as shown in Listing 7. This makes it possible to react to dynamically changed values and use them in application logic.

```
const root          = document.documentElement;
const contrastColor = getComputedStyle(root).getPropertyValue('--
contrast-color');
```

Listing 7: Reading a CSS custom property in JavaScript using `getPropertyValue()`.

4.2.1. Global State Management

CSS custom properties are not limited to storing design values such as colors. They can also be used to manage application state. Because custom properties can be read and modified in both CSS and JavaScript, they provide a convenient way to store configuration values that directly influence the user interface.

For example, a user may not have enabled reduced motion at the operating system or browser level, but may choose to disable animations through a website-specific

setting. This preference can be stored in a custom property such as `--prefers-reduced-motion: true`. The `@container` at-rule can then react to this value and apply different styles at runtime, as shown in Listing 8. [18]

```
:root {
  --prefers-reduced-motion: false;
}

@container style(--prefers-reduced-motion: true) {
    .animated-contents {
        animation: none !important;
    }
}
```

Listing 8: Using the CSS `@container` at-rule to disable animations based on a custom property.

Another approach is to use a CSS attribute selector to override custom properties when a different color theme is selected. For example, the operating system and browser may indicate a preference for a light theme, while the user explicitly selects a dark theme on the website. In this case, a custom property such as `--prefers-dark-theme: true` can be used to switch the application's color palette, as demonstrated in Listing 9. [19]

```
:root {
  --prefers-dark-theme: false;

  --text-color:     #222222;
  --bg-color:       #fcfcfc;
  --border-color:   #eeeeee;
  --primary-accent: #0056b3;
}

:root[style*="--prefers-dark-theme: true"] {
  --text-color:     #ededed;
  --bg-color:       #212121;
  --border-color:   #232323;
  --primary-accent: #6Db3f4;
}
```

Listing 9: Using a CSS attribute selector to switch to a dark color theme based on a custom property.

4.2.2. Custom Property Toggle

In modern CSS, developers often face the challenge of managing repetitive declarations, especially when implementing light and dark themes. Whether the switch is handled through classes or through `@media (prefers-color-scheme: dark)`, the same property names typically need to be declared multiple times with different values. A lesser-known detail in the CSS specification makes it possible to implement this kind of state management more elegantly by using custom properties as toggle values.

According to the CSS specification, a custom property must contain at least one token to be considered valid. A single whitespace character satisfies this requirement and can therefore be used to simulate boolean-like values, as shown in Listing 10. [20]

```
:root {
  --ON: ;
  --OFF: initial;
}
```

Listing 10: Both `' '` and `initial` are valid values for custom properties.

In traditional theming setups, each variable must be defined separately for both the light and dark theme. By using the pseudo-boolean values introduced in Listing 10, the theme state can be stored in a single custom property, as demonstrated in Listing 11.

```
:root {
  --is-light-theme: var(--ON);
}

@media (prefers-color-scheme: dark) {
  :root {
    --is-light-theme: var(--OFF);
  }
}
```

Listing 11: Assigning either `' '` or `initial` depending on the active theme creates the basis for the toggle logic.

When `--is-light-theme` is used as a prefix in another custom property declaration, the resulting declaration is either valid or invalid depending on its value. If the prefix expands to a whitespace character, the declaration remains valid. If it expands to

`initial`, the declaration becomes invalid. In that case, the fallback value provided to `var()` is used automatically, as illustrated in Listing 12.

```
:root {
  --color-text--light: var(--is-light-theme) black;
  --color-text--dark: white;

  --color-background--light: var(--is-light-theme) white;
  --color-background--dark: black;

  --color-text: var(--color-text--light,
                var(--color-text--dark));
  --color-background: var(--color-background--light,
                      var(--color-background--dark));
}
```

Listing 12: When a custom property becomes invalid, `var()` automatically uses the fallback value.

Multiple fallback values based on different toggle properties can be chained using the CSS `var()` function. This enables centralized state management, where the custom properties used throughout the entire style sheet are controlled from a single location. [21], [22]

An example of this theme toggle is available in the project's repository: https://github.com/Flaverus/P8_sandbox/tree/main/examples/theme-switch

4.3. CSS Functions

CSS provides a wide range of built-in functions that simplify common styling tasks. Some of these functions are particularly useful for accessibility. One notable example is `contrast-color()`, which became broadly available during the implementation of this project in April 2026 and is supported by current browser versions. The function returns either black or white, depending on which of the two provides the higher contrast against the color passed as its argument. Listing 13 shows this approach in the context of a `button` element. [23]

```
button {
  background-color: var(--button-color);
  color: contrast-color(var(--button-color));
}
```

Listing 13: Setting the text color of a button to black or white based on the background color.

Using either black or white for text is a reliable way to ensure strong contrast. Depending on the background color, however, the result may appear visually harsh and less pleasant to read. Another recently introduced CSS function is `light-dark()`, which accepts two colors or images. The first value is used when a light color scheme is active, while the second value is used for a dark color scheme. Listing 14 demonstrates this approach using custom properties. [24]

```
body {
  color: light-dark(var(--color-text--light),
                    var(--color-text--dark));
  background-color: light-dark(var(--color-background--light),
                               var(--color-background--dark));
}
```

Listing 14: Defining text and background colors based on the active color theme using custom properties.

4.3.1. CSS Custom Functions

Although the growing set of built-in CSS functions covers many use cases, some scenarios still require functionality that cannot be expressed with the currently available primitives. In such cases, developers may need to combine multiple functions or implement custom logic. This is where the experimental `@function` at-rule becomes relevant.

This feature is conceptually similar to CSS custom properties, as custom functions also use names that begin with `--`. A function is declared using the `@function` keyword, followed by a custom name and an optional list of parameters. Both the function name and its parameters start with `--` and are followed by case-sensitive identifiers defined by the developer. Inside the function body, the expression assigned to `result:` is evaluated and returned, as shown in Listing 15. [25]

```
@function --color-contrast(--color <color>) returns <color> {
  result: oklch(from var(--color) calc((0.5 - l) * infinity) 0 0);
}
```

Listing 15: A custom function that provides behavior similar to `contrast-color()` and was created before that function became widely available.

As with CSS custom properties, CSS data types such as `<color>`, `<number>`, and `<string>` can be specified for both function parameters and the return value.

4.4. View Transition API

The View Transition API provides a native mechanism for animating transitions between different states of a web application. It can be used to create smooth visual effects when navigating between pages or when updating views within single-page applications (SPAs).

For transitions that occur within the same document, the state change is wrapped inside the `document.startViewTransition()` method. Cross-document transitions in multi-page applications (MPAs) are triggered automatically during navigation and can be enabled through the `@view-transition` at-rule. Internally, the API captures snapshots of both the previous and the new state and animates between them. By default, this transition consists of a simple fade effect, where the old view gradually decreases its opacity to `0` while the new view increases its opacity to `1`.

The generated animations can be customized through the `::view-transition-old()` and `::view-transition-new()` pseudo-elements, which target the previous and new view respectively. Shared styling can be applied through `::view-transition-group()`. These pseudo-elements accept different targets, including `root` for animating the entire page, `*` for all transition targets, or a custom `view-transition-name` to animate specific elements independently.

An example of a custom page transition is shown in Listing 16. In this case, the previous page is animated out of view towards the left while the new page enters from the right, creating a horizontal swipe effect. [26]

```
@keyframes move-out {
  from {
    transform: translateX(0%);
  }
  to {
    transform: translateX(-100%);
  }
}

@keyframes move-in {
  from {
    transform: translateX(100%);
  }
  to {
    transform: translateX(0%);
  }
}

::view-transition-old(root) {
  animation: 0.4s ease-in both move-out;
}

::view-transition-new(root) {
  animation: 0.4s ease-in both move-in;
}
```

Listing 16: A custom page transition that swipes the previous view out to the left while moving the new view in from the right.

4.5. Color Spaces

There are many different models for describing the colors perceptible to the human eye. Each model has its own strengths and weaknesses and is used in different contexts, ranging from print and photography to digital displays.

For many years, CSS relied primarily on the RGB color space, which limited how colors could be described and manipulated. To update features faster, the W3C now evolves CSS through independent modules rather than giant versions. Under this system, the CSS Color Module Level 4 specification acts as a direct software update to the three previously released levels, introducing support for additional color spaces and more advanced color functions to significantly expand the possibilities for working with color in CSS. Development in this area continues, and the W3C is currently working on the CSS Color Module Level 5 specification.

4.5.1. RGB

The RGB color model represents colors by combining different intensities of the primary colors `red`, `green`, and `blue`. An optional alpha channel can be added to define the opacity of the resulting color.

RGB has been supported in CSS since its early days and can be expressed using the `rgb()` function or hexadecimal notation. For example, the color shown in Listing 17 can also be written as `#2d1fb180`. [27]

```
rgb(45 31 177 / 0.5);
```

Listing 17: Using `rgb()` to define a color from the Kolibri palette with 50% opacity.

4.5.2. HSL

CSS Color Module Level 3 introduced the HSL color model to CSS. HSL describes colors using three components: `hue`, `saturation`, and `lightness`, as illustrated in Listing 18. The hue is represented as an angle on the color wheel, while saturation and lightness define the intensity and brightness of the color. As with RGB, an optional alpha channel can be added to control opacity.

HSL makes it easier to create variations of a color because saturation and lightness can be adjusted directly without recalculating individual RGB values. This is particularly useful when generating lighter or darker shades of the same base color. [27]

```
hsl(256 82 55 / 0.6);
```


Listing 18: Using `hsl()` to define a color from the Kolibri palette with 60% opacity.

4.5.3. CIELAB Colors

Perceptual color spaces such as Oklab and OkLCH make it possible to define colors in a way that more closely matches human vision. They also provide access to a wider range of colors than those that can be represented in the traditional sRGB color space.

The `oklab()` function describes a color using three components: `lightness`, the `a` axis representing red-green variation, and the `b` axis representing yellow-blue variation. The related `oklch()` function uses `lightness`, `chroma`, and `hue` instead. Both functions support an optional alpha channel to control opacity. An exemplary usage of both functions can be observed in Listing 19.

These color models are widely regarded as the current state of the art for working with color in CSS because they produce more perceptually uniform results. In practice, this means that adjusting lightness or chroma leads to more predictable visual changes than with older color models. [27]

```
oklab(0.638 0.176 -0.279 / 0.7);
oklch(0.718 0.255 301.5 / 0.7);
```


Listing 19: Using `oklab()` and `oklch()` to define a color from the Kolibri palette with 70% opacity.

4.5.4. HWB

The `hwb()` color function was introduced alongside the newer CSS color features. It describes colors using three components within the RGB color space: `hue`, `whiteness`, and `blackness`. In practice, it offers an alternative to `hsl()` that many developers find more intuitive when adjusting tints and shades. An example is shown in Listing 20. [27]

```
hwb(325 18 0 / 0.3);
```


Listing 20: Using `hwb()` to define a color from the Kolibri palette with 30% opacity.

4.6. Contrast Perception

Contrast is essential for distinguishing elements and understanding their relationships. Differences in brightness, size, texture, and shape all contribute to visual contrast. On the web, variations in lightness are the primary means of ensuring text readability and clearly separating interface elements.

The following section introduces the contrast requirements defined in WCAG 2.x and provides an overview of the emerging approach proposed for WCAG 3.0.

4.6.1. Relative Luminance

The contrast requirements in WCAG 2.x are based on the relative luminance of text and its background, independent of hue. This approach assumes that hue and saturation have little influence on reading performance, even for users with color vision deficiencies. Since the inability to distinguish certain colors does not usually affect the perception of light and dark, color itself is not treated as a primary factor in the contrast calculation.

In practice, the perceived contrast may differ from the theoretical value. Smaller or thinner fonts can appear noticeably lighter than the color specified in CSS due to anti-aliasing and font rendering. As a result, text may appear to have lower contrast than the calculated ratio suggests.

WCAG 2.x requires a minimum contrast ratio of 4.5:1 for normal text to meet Level AA. Large text must reach at least 3:1. For Level AAA, the minimum contrast ratio increases to 7:1. These thresholds are based in part on recommendations from ISO 9241-3 and are intended to compensate for reduced contrast sensitivity in users with moderate visual impairments. Users with more severe vision loss typically rely on assistive technologies such as screen magnifiers or screen readers.

Color vision deficiencies vary significantly, making it difficult to define universally effective color combinations based solely on hue. This is one of the main reasons why WCAG evaluates contrast primarily through relative luminance.

One notable exception is `protanopia`, in which red tones may appear significantly darker than expected. As a result, red text on dark backgrounds can provide much less contrast than the calculated ratio suggests and should generally be avoided. [28]

The formula shown in Listing 21 is used by WCAG 2.x to calculate relative luminance, which serves as the basis for determining contrast ratios. [29]

$$L = 0.2126 \cdot R + 0.7152 \cdot G + 0.0722 \cdot B$$

$$\text{Where } \; C = \begin{cases} \frac{C_{\text{sRGB}}}{12.92} & \text{if } C_{\text{sRGB}} \leq 0.03928 \\ \left(\frac{C_{\text{sRGB}}+0.055}{1.055}\right)^{2.4} & \text{otherwise} \end{cases}$$

Example: Dodger Blue `rgb(30, 144, 255)`

1. Normalize (/255)
$R_{\text{sRGB}} = \frac{30}{255} = 0.1176$
$G_{\text{sRGB}} = \frac{144}{255} = 0.5647$
$B_{\text{sRGB}} = \frac{255}{255} = 1.0000$

2. Linearize
$R = 0.0123$
$G = 0.2747$
$B = 1.0000$

3. Calculate Luminance (L)

$$L = (0.2126 \cdot 0.0123) + (0.7152 \cdot 0.2747) + (0.0722 \cdot 1.0000) = \mathbf{0.2712}$$

Calculate Contrast Ratio

$$\text{Ratio} = \frac{\text{L1} + 0.05}{\text{L2} + 0.05}$$

Listing 21: Step-by-step calculation of relative luminance based on the W3C sRGB formula. [29]

4.6.2. Perceptual Color Models

The contrast model used in WCAG 2.x is based on relative luminance. In the future, it is expected to be replaced by the Accessible Perceptual Contrast Algorithm (APCA), which forms part of the ongoing WCAG 3.0 work. Advances in display technology, modern web design, and vision science have shown that the approach introduced nearly two decades ago in WCAG 2.x no longer reflects how contrast is perceived in practice.

WCAG 2.x follows a binary model in which a contrast ratio either passes or fails. APCA takes a different approach by accounting for the non-linear nature of human perception and by evaluating contrast in the context in which the colors are used.

Readability depends on more than the luminance difference between two colors. Font size, font weight, stroke thickness, and surrounding context all influence how contrast is perceived. High light-dark contrast generally improves readability, but smaller and thinner text requires a greater difference in lightness to remain legible, as illustrated in Figure 1.

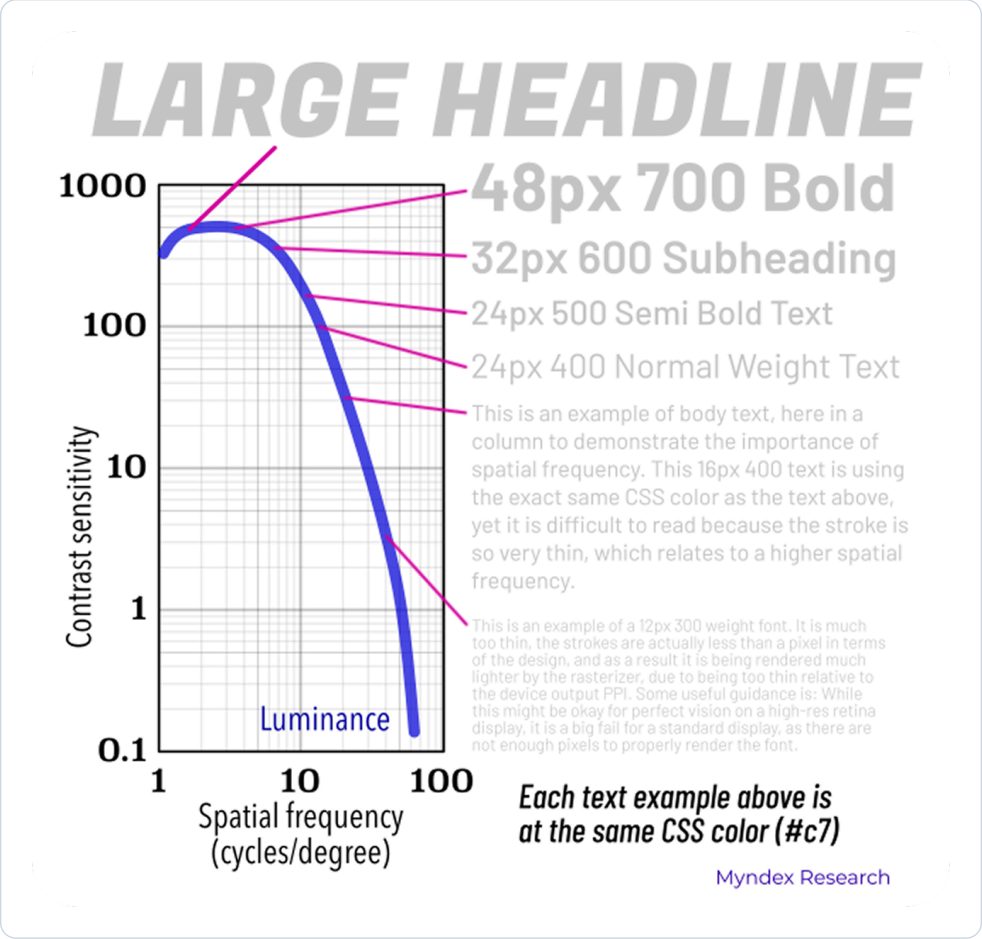

Figure 1: This chart illustrates the spatial dependence of human contrast sensitivity using text samples. [30]

Non-text elements such as icons typically require less contrast than body text. More generally, the appropriate contrast between two colors depends on the specific use case, including the size, thickness, and purpose of the element.

The contrast formula used in WCAG 2.x has been criticized for many years. Case studies comparing white and black text on colored backgrounds have shown that

combinations classified as inaccessible under WCAG 2.x were sometimes perceived as more readable by participants with color vision deficiencies. [31]

APCA introduces a new metric called lightness contrast, expressed as L^c. Instead of assigning a universal pass-or-fail threshold, it estimates readability based on the perceptual relationship between foreground and background colors within their actual visual context. [30]

4.7. Canvas

The HTML `<canvas>` element is used to draw graphics and animations directly within a web page. By default, the element has no visual representation of its own and must be rendered through JavaScript using either the `Canvas API` or the `WebGL API`.

The `Canvas API` is primarily designed for two-dimensional graphics and is commonly used for tasks such as game rendering, data visualization, image manipulation, and real-time video processing. The `WebGL API` extends these capabilities by providing hardware-accelerated rendering for both two-dimensional and three-dimensional graphics.

In addition to drawing content, the canvas also allows pixel data to be read and analyzed. The `ImageData` interface provides access to the underlying RGBA values of a rendered image. Listing 22 demonstrates how the color values of a selected area can be extracted in JavaScript. [32], [33], [34]

```
const canvas = document.createElement('canvas');
const ctx    = canvas.getContext('2d');

ctx.fillStyle = 'oklab(0.638 0.176 -0.279)';
ctx.fillRect(0, 0, 100, 100);

const red   = ctx.getImageData(10, 10, 10, 10).data[0];
const green = ctx.getImageData(10, 10, 10, 10).data[1];
const blue  = ctx.getImageData(10, 10, 10, 10).data[2];
const alpha = ctx.getImageData(10, 10, 10, 10).data[3];
```

Listing 22: Creating a purple 100 × 100 pixel square and extracting the RGBA values from a 10 × 10 pixel region starting at coordinates $x = 10$ and $y = 10$.

4.8. Ishihara

The Ishihara color test, named after the Japanese ophthalmologist Shinobu Ishihara, was developed in 1917 to detect red-green color vision deficiencies. The test consists of a series of circular plates composed of pseudo-isochromatic dots. These dots vary in size and color and are arranged in patterns that appear distinct to individuals with typical color vision, while blending together for people with certain forms of color blindness. [35]

An example of such a plate is shown in Figure 2. It uses orange and teal tones and is designed to remain visible regardless of the viewer's color perception. Plates of this kind are commonly used as introductory demonstration images at the beginning of an Ishihara test.

4.8.1. Functional Application in Contrast Testing

Although originally developed as a diagnostic tool, the underlying principle of Ishihara plates also provides a useful framework for evaluating color contrast in interface design.

By placing many small dots of different colors next to each other, the method forces the visual system to rely primarily on chromatic contrast to identify a pattern. In a custom accessibility tool, if the foreground and background colors visually blend together within such a plate, this indicates that the chosen color combination may not provide sufficient perceptual contrast.

Custom plates can be generated using a specific brand palette to verify that key colors remain distinguishable not only for users with typical color vision, but also for users with conditions such as `protanopia` or `deuteranopia`.

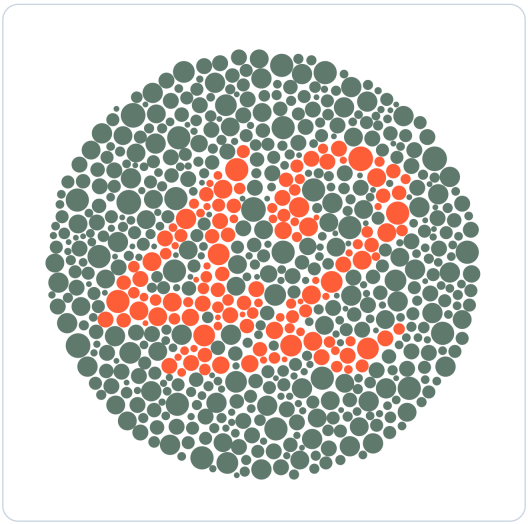

Figure 2: A high-contrast demonstration plate designed to remain legible across all common vision types.

5. Methodology

5.1. Research and Ideation

This project initially started in a different direction by exploring how accessibility can be tested in an automated manner. During this research phase, a wide range of accessibility testing tools was examined, including both free and commercial solutions. One particularly influential source was the blog of Karl Groves, which states that only around 30–40% of accessibility issues can be reliably detected through automated testing [36]. Although the original article was published in 2012, it received updates in late 2025. While automated accessibility testing has improved since the article was first written, the research process gradually shifted the focus of this project into another direction.

During this phase, additional ideas emerged regarding accessibility support beyond testing itself. Instead of focusing exclusively on automated evaluation, the project evolved toward exploring ways to enhance browser standards and provide additional accessibility aids directly within web applications. Since accessibility testing is already heavily researched and commercially established, this alternative direction appeared more valuable and interesting for further exploration.

After redefining the project goals, an exploratory prototyping approach was chosen to rapidly test and refine ideas over time. Existing standards, emerging browser capabilities, and newly introduced CSS and JavaScript features were researched to build a broad overview of current possibilities.

Browser support and compatibility were considered throughout the process to better understand both practical limitations and future opportunities. Experimental features and unconventional approaches were intentionally included where appropriate to avoid building solutions solely around already outdated technologies and standards.

5.2. Prototyping

The first practical step was creating an isolated environment, referred to as the Sandbox project, for rapid prototyping and experimentation. This environment allowed different implementation approaches to be tested incrementally while evolving the individual project parts over time.

Testing CSS and JavaScript compatibility directly in practice proved especially important, as documentation alone often did not fully reflect actual browser behavior. Experimentation and playful exploration became a major part of the ideation process before more concrete concepts gradually emerged and were refined into the implementations presented throughout this documentation.

Many examples initially started as simple or partially hacky drafts before being iteratively improved into more stable and reusable solutions.

5.3. Mathematical Foundation

To develop a deeper understanding of the current WCAG 2.x contrast standard, the W3C documentation regarding relative luminance and RGB color calculations was studied extensively. This research helped clarify the mathematical relationship between RGB color values, luminance calculation, and contrast ratios.

The resulting understanding made it possible to formalize the algorithmic approach described in the specification into a clearer mathematical representation. This later enabled the implementation of a contrast ratio indicator that was used within the custom contrast color function example application.

This process also motivated further research into the future WCAG 3.0 contrast model. The WCAG 2.x documentation itself acknowledges known limitations in the current contrast formula, including simplifications, rounding inaccuracies, and shortcomings regarding human visual perception. During this research, the APCA model was identified as a promising future replacement. However, because APCA is still under active development and currently remains in draft status, it was not integrated further into the practical implementations of this project.

5.4. Validation and Verification

The Preferences Widget underwent smaller-scale user testing to gather feedback from real users and refine the interface for future integration into Kolibri. The tests were intentionally kept simple to allow participation from users with different backgrounds and experience levels. The primary focus was usability, keyboard accessibility, intuitiveness, and the overall visual experience.

The different example applications and prototype pages were additionally validated using automated accessibility testing tools such as the axe DevTools browser extension and Google Lighthouse. These automated checks were supplemented with manual keyboard accessibility testing to verify practical usability beyond purely automated evaluation.

6. Implementation

This chapter presents practical approaches that extend existing browser standards to further improve accessibility. The **preferences widget** enables users to define website-specific settings tailored to their individual needs. The **contrast color function** expands the native `contrast-color()` CSS function by introducing an additional color component. The **interactive Ishihara plate** allows users to evaluate color combinations manually and to test contrast under simulated color vision deficiencies.

6.1. Preferences Widget

CSS media queries already make it possible to respond to certain user preferences. However, this only works when the user has configured the corresponding settings in the operating system or browser. If no such settings exist, websites have no reliable way to adapt to the user's individual accessibility needs.

Even when system-level preferences are configured, they may not reflect the requirements of every website. A user may prefer reduced motion in general, but still want to enable animations on a specific page, or the opposite. In addition, some accessibility preferences, such as color blindness simulations, are not currently covered by CSS media queries and therefore cannot be addressed through native browser mechanisms alone.

To overcome these limitations and give users greater control, the preferences widget shown in Figure 3 was developed. It uses existing system settings as a starting point, while allowing them to be adjusted and extended on a per-website basis.

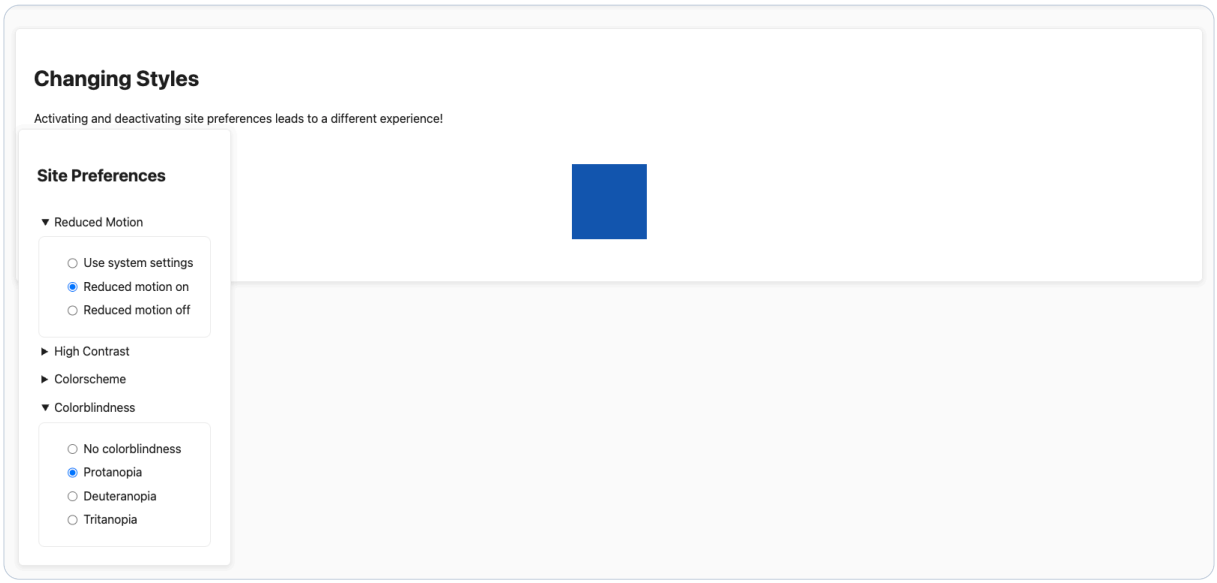

Figure 3: A screenshot of the preferences widget with page-specific settings for reduced motion and color blindness.

6.1.1. JavaScript Part

Where supported, existing CSS media queries serve as the foundation for the widget's default values. When a user visits the page for the first time, the current operating system or browser settings are used as the initial configuration. Any changes made through the widget are stored in the browser's `localStorage`, ensuring that preferences persist across page reloads and remain available throughout the application.

Each preference is also written to a CSS custom property on the `:root` element, making the value accessible from both JavaScript and CSS. This approach establishes a single source of truth that can be used consistently across the entire application, as shown in Listing 23.

```
const setAccessibilityProperty = (property,
                                  value,
                                  mediaQueryString) => {

  localStorage.setItem(property, value);

  if(value === 'system' && mediaQueryString !== '') {
    const systemSetting = window.matchMedia(mediaQueryString).matches;
    root.style.setProperty(property, systemSetting);
  } else {
    root.style.setProperty(property, value);
  }
}
```

Listing 23: The `setAccessibilityProperty()` function stores a value in `localStorage` and updates the corresponding custom property on `:root`.

The `getAccessibilityProperty()` function shown in Listing 24 reads a previously saved value from `localStorage`. If no value has been stored yet, it returns `'system'`, indicating that the operating system or browser setting should be used as the default.

```
const getAccessibilityProperty = (property) => {
    const savedProperty = localStorage.getItem(property);

    if(savedProperty) {
        return savedProperty;
    } else  {
        return 'system';
    }
}
```

Listing 24: The `getAccessibilityProperty()` function returns a saved value or falls back to the system setting.

During page initialization, both functions are used to populate the custom properties with either the stored value or the corresponding system preference, as shown in Listing 25.

```
setAccessibilityProperty('--prefers-reduced-motion',
                         getAccessibilityProperty(
                             '--prefers-reduced-motion'),
                         '(prefers-reduced-motion: reduce)');
setAccessibilityProperty('--prefers-contrast',
                         getAccessibilityProperty(
                             '--prefers-contrast'),
                         '(prefers-contrast: more)');
setAccessibilityProperty('--prefers-dark-theme',
                         getAccessibilityProperty(
                             '--prefers-dark-theme'),
                         '(prefers-color-scheme: dark)');
setAccessibilityProperty('--prefers-colorblind-mode',
                         getAccessibilityProperty(
                             '--prefers-colorblind-mode'),
                         '');
```

Listing 25: Initializing the global custom properties with stored values or system defaults.

The `syncWidgetOption()` function shown in Listing 26 synchronizes the values stored in the custom properties with the corresponding radio buttons in the widget. By reading the values directly from `:root`, the custom properties remain the single source of truth for both the user interface and the underlying logic.

```
const syncWidgetOption = (option, property) => {
  const value = getAccessibilityProperty(property);

  option.forEach(radio => {
    radio.checked = radio.value === value;
  });
}

syncWidgetOption(motion,         '--prefers-reduced-motion');
syncWidgetOption(contrast,       '--prefers-contrast');
syncWidgetOption(colorscheme,    '--prefers-dark-theme');
syncWidgetOption(colorblindness, '--prefers-colorblind-mode');
```

Listing 26: Synchronizing the selected radio buttons with the values stored in the custom properties.

Finally, each radio group registers a `change` event listener that updates both `localStorage` and the corresponding custom property whenever the user selects a different option, as shown in Listing 27.

```
addOptionEventListener(motion,
                       '--prefers-reduced-motion',
                       '(prefers-reduced-motion: reduce)');
addOptionEventListener(contrast,
                       '--prefers-contrast',
                       '(prefers-contrast: more)');
addOptionEventListener(colorscheme,
                       '--prefers-dark-theme',
                       '(prefers-color-scheme: dark)');
addOptionEventListener(colorblindness,
                       '--prefers-colorblind-mode',
                       '');

const addOptionEventListener = (option, property, mediaQueryString) =>
{
  option.forEach(radio => {
    radio.addEventListener('change', () => {
      if(radio.checked) {
        setAccessibilityProperty(property, radio.value,
mediaQueryString);
      }
    });
  });
}
```

Listing 27: Updating the user's preference when the selected radio button changes.

6.1.2. CSS Part

Once the CSS custom properties representing the user's preferences are in place, they can be used to create different visual representations of the website. In Listing 28, a set of custom properties defines the default styling. These values are selectively overridden using the CSS attribute selector `[]` when the user prefers a dark color scheme.

```css
:root {
  --bg-color:       #fcfcfc;
  --text-color:     #222222;
  --content-bg:     #ffffff;
  --border-color:   #eeeeee;
  --primary-accent: #0056b3;
  --shadow:         0 2px 8px rgba(0, 0, 0, 0.1);

  color-scheme: light;
}

:root[style*="--prefers-dark-theme: true"] {
  --bg-color:       #212121;
  --text-color:     #ededed;
  --content-bg:     #2e2e2e;
  --border-color:   #232323;
  --primary-accent: #6db3f4;
  --shadow:         0 2px 8px rgba(0, 0, 0, 0.5);

  color-scheme: dark;
}
```

Listing 28: Overriding the default custom properties with the CSS attribute selector on `:root` to apply a dark color scheme.

As additional preferences are introduced, the number of possible combinations increases. Listing 29 demonstrates how the base styling is adjusted when both the dark theme and the high contrast mode are enabled, resulting in a different visual presentation.

```
:root[style*="--prefers-dark-theme: true"]
[style*="--prefers-contrast: true"] {
  --bg-color:       #000000;
  --text-color:     #ffffff;
  --content-bg:     #000000;
  --border-color:   #ffffff;
  --primary-accent: #80c5ff;
  --shadow:         none;
}
```

Listing 29: Applying a dedicated style when both dark mode and high contrast mode are enabled.

In some cases, changing individual custom property values is not sufficient and additional CSS rules must be applied. The `@container` at-rule combined with the `style()` query makes this possible. Because the preference properties are defined on `:root`, they can be used to conditionally apply styles across the entire application. Listing 30 demonstrates how animations and transitions can be disabled when the user prefers reduced motion.

```
@container style(--prefers-reduced-motion: true) {
  .animated-contents {
    animation: none !important;
  }

  .transitional-contents {
    transition: none !important;
  }
}
```

Listing 30: Applying additional styles based on custom properties using the `@container` rule and a `style()` query.

6.1.3. The Paradox of Reduced Motion Transitions

An interesting and somewhat ironic aspect is that enabling reduced motion preferences does not necessarily mean that all transitions and animations should be removed from a webpage. In certain contexts, the opposite can even be beneficial. Different accessibility concerns require different solutions, and some of these measures may appear counterintuitive at first glance.

For example, users who prefer reduced motion may still benefit from smooth and controlled transitions when switching between color themes. Abrupt flashes or immediate color changes, even when triggered intentionally by the user, can create

discomfort or visual strain. In such situations, introducing subtle transitions may improve the overall experience rather than worsen it.

Although it may initially seem contradictory to apply transitions when `--prefers-reduced-motion: true` is configured, carefully controlled color transitions can therefore represent a valid accessibility measure.

6.1.3.1. Animating Properties

To animate CSS custom properties, the `@property` at-rule is required. This informs the browser about the expected value type of a property, such as `<color>`, allowing it to interpolate the values during transitions. Once defined, transitions can be applied directly to the custom properties within a `@container` rule using the global `*` selector, since the registered properties themselves are not scoped locally. Listing 31 demonstrates how custom color properties can be registered and transitioned.

```
@property --bg-color {
  syntax: '<color>';
  inherits: true;
  initial-value: #fcfcfc;
}

@property --text-color {
  syntax: '<color>';
  inherits: true;
  initial-value: #222222;
}

@container style(--prefers-reduced-motion: true) {
  * {
    transition: --bg-color   0.3s linear,
                --text-color 0.3s linear,
  }
}
```

Listing 31: Having custom properties defined with `@property` to be able to add a transition within a `@container` rule.

6.1.3.2. View Transition

While animating individual properties can be useful, it may result in less coherent transitions when multiple properties change simultaneously. In such situations, individual animations can compete for attention and create a visually noisy effect that is ultimately more distracting than beneficial. To avoid this, it is often preferable to transition the page state as a whole by using the View Transition API.

For the Preferences Widget, this can be achieved by wrapping the property update inside the `document.startViewTransition()` method, as shown in Listing 32. This allows the browser to capture the previous and new state of the page and animate the transition between them in a coordinated manner. The resulting animation can be further refined through CSS. For example, the duration can be increased to create a smoother transition, as demonstrated in Listing 33.

```
document.startViewTransition(() => {
  root.style.setProperty(property, appliedValue);
});
```

Listing 32: Enabling a view transition for a property change using the `startViewTransition()` method.

```
::view-transition-group(root) {
    animation-duration: 0.6s;
}
```

Listing 33: Increasing the duration of the view transition to create a smoother visual effect.

The implementation of the preference widget, including several example settings that demonstrate how individual and combined preferences affect the website's styling, is available in the project's repository: https://github.com/Flaverus/P8_sandbox/tree/main/examples/widget

6.2. Contrast Color Function

The theory chapter introduced the native `contrast-color()` CSS function and demonstrated how similar behavior can be implemented using CSS custom functions. Given a color, `contrast-color()` returns either black or white, depending on which of the two provides the greater lightness contrast. While this approach is effective, pure black and white can appear visually harsh, especially when used for larger blocks of text.

To address this limitation, an enhanced version of `contrast-color()` was developed during this project and is shown in Listing 34. The custom function accepts a required parameter of type `<color>` and an optional `<percentage>` parameter named `--intensity`.

In the first step, the function determines whether black or white provides the better contrast by evaluating the lightness component of the input color in the OkLCH color space. This is achieved by subtracting the lightness value from `0.5`, representing 50% lightness, and multiplying the result by the constant `infinity`. Due to the way `oklch()` handles lightness values, this effectively collapses the result to either 0 or 1, depending on whether the original color is darker or lighter than 50%. If the lightness value is greater than or equal to 50%, black is selected. Otherwise, white is used.

In the second step, the selected contrast color is passed to the `color-mix()` function. The optional `--intensity` parameter controls how much of the original color is mixed back into the result. This produces a softer contrast color that retains some of the visual characteristics of the source color.

To further reduce extreme contrast, the lightness of the selected black or white is adjusted using the `clamp()` function. This ensures that light colors do not exceed a lightness of 97.5% and dark colors do not fall below 15%. As a result, the generated contrast color remains readable while appearing less visually aggressive. The selected bounds represent pragmatic perceptual limits intended to preserve strong readability while reducing the visual harshness associated with maximum contrast combinations.

```
@function --color-contrast(--color <color>, --intensity <percentage>:
0%) returns <color> {
  --black-or-white: oklch(from var(--color) calc((0.5 - l) * infinity)
0 0);

  result: color-mix(in oklch, oklch(from var(--black-or-white)
clamp(0.15, l, 0.975) c h), var(--color) var(--intensity));
}
```

Listing 34: A custom contrast function that softens pure black and white and optionally mixes in a portion of the original color.

Note: The specific thresholds of 15% and 97.5% used in this function serve as a subjective, baseline example to illustrate the concept. These values are not absolute standards and should be modified to align with the specific contrast ratios, aesthetic preferences, and accessibility requirements of the design system in use.

The accompanying example from the project's example collection also calculates the WCAG 2.x contrast ratio for the generated colors, as shown in Figure 4.

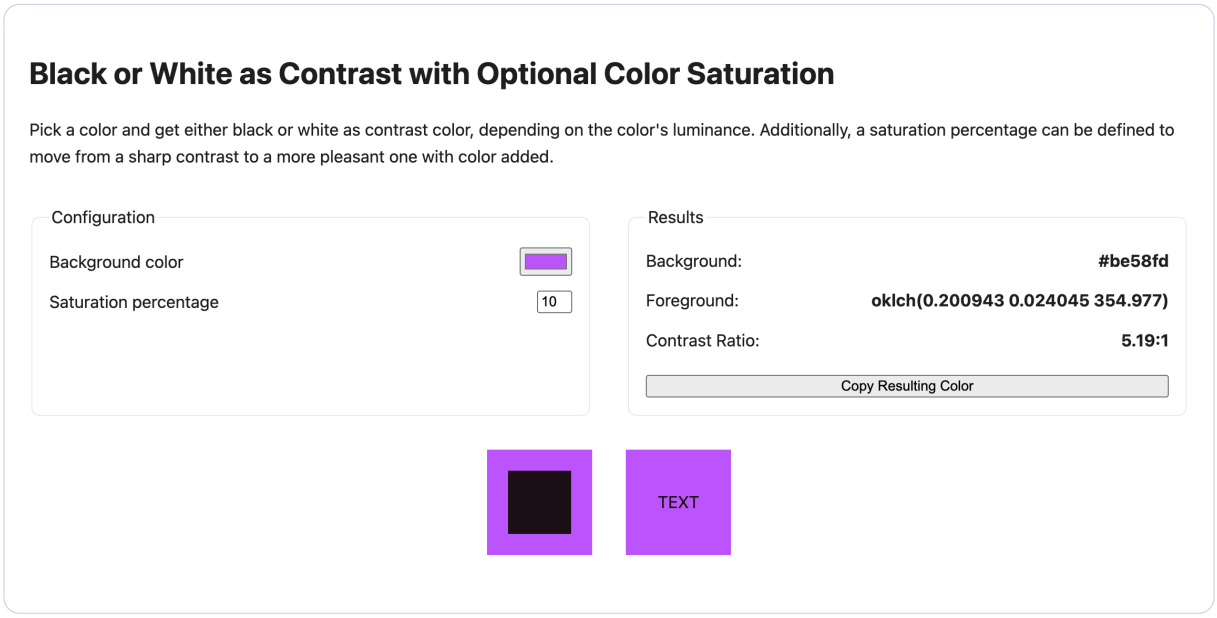

Figure 4: A screenshot of the configuration interface for the custom contrast color function.

To calculate the WCAG 2.x contrast ratio, the RGB values of both colors are required. When colors are defined using functions such as `oklch()`, these values are not directly available, as browsers may preserve the original color format in the computed styles.

A practical workaround is to use the HTML `<canvas>` element. The canvas can be filled with any valid CSS color, and the resulting RGBA values can then be extracted using `getImageData()`. As shown in Listing 35, this technique makes it possible to convert any supported CSS color format into numeric RGBA values.

```
const parseColorToRGBA = (colorString) => {
  const canvas = document.createElement('canvas');
  const ctx    = canvas.getContext('2d');

  ctx.fillStyle = colorString;
  ctx.fillRect(0, 0, 1, 1);

  // Returning data array contining RGBA values as follows: r, g, b, a
  return ctx.getImageData(0, 0, 1, 1).data;
};
```

Listing 35: Extracting the `RGBA` values of any CSS color using the HTML `<canvas>` element.

The implementation of the custom contrast color function is available in the project's repository: https://github.com/Flaverus/P8_sandbox/tree/main/examples/contrast-color

6.3. Interactive Ishihara Plate

The interactive Ishihara plate is not intended to directly enhance a website through integration into a production environment. Instead, it serves as a developer tool for visually evaluating color contrast in situations where shape, placement, or additional visual cues do not influence recognition. The focus lies entirely on the perception of color itself.

The application allows three separate colors to be defined for the background circles and three additional colors for the foreground circles. Through their arrangement, the foreground circles form the number 42. This setup makes it possible to evaluate how different shades of the same color, for example variations in saturation or lightness, interact with one another and whether sufficient visual contrast remains between foreground and background elements. An example configuration using colors from the Kolibri palette is shown in Figure 5.

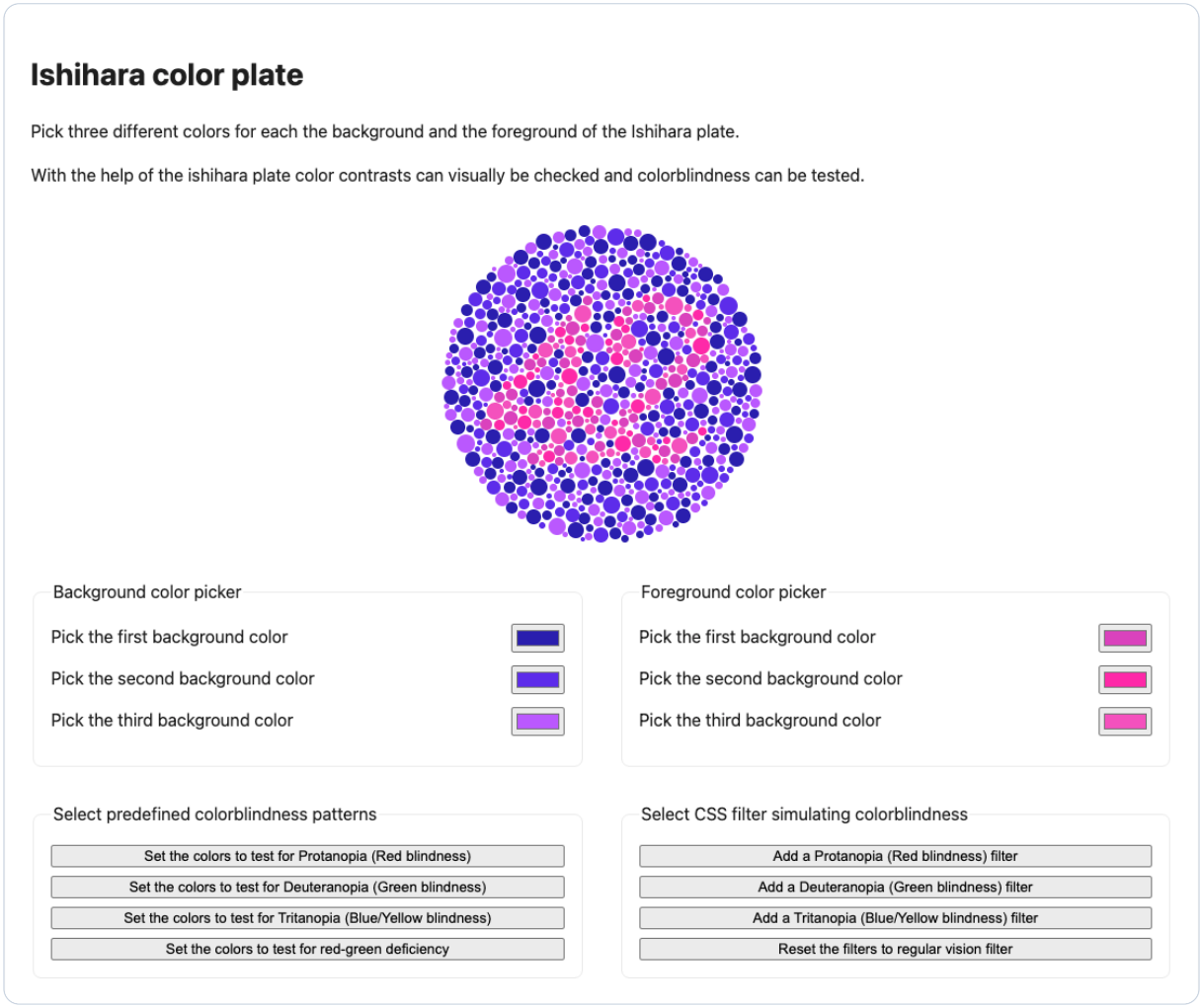

Figure 5: A screenshot of the interactive Ishihara plate configured with colors from the Kolibri palette.

In addition to freely configurable colors, the application also includes predefined color combinations designed to simulate scenarios that are difficult or impossible to distinguish for people with specific forms of color vision deficiency such as `Protanopia`, `Deuteranopia`, and `Tritanopia`. These presets make it possible to evaluate whether certain color combinations remain distinguishable under different forms of impaired color perception.

Although these configurations are inspired by the original Ishihara test plates, the application is not intended to serve as a medically accurate diagnostic tool. Instead, it should be considered a visual indicator that may suggest the need for further professional examination.

An example of these comparison modes can be seen in Figure 6. The left side displays a plate configured with colors that are difficult to distinguish for users with `Deuteranopia`. The right side shows the same plate with a `Deuteranopia` simulation filter applied, illustrating how the color combination may appear to affected users. The simulation filters are based on the bookmarklet filters developed during the previous P7 project.

A more detailed discussion of these bookmarklets is available in the corresponding chapter of the previous P7 project: https://accessible-web-initiative.gitbook.io/accessibility-on-the-web-where-we-stand/research/debugging-and-testing-accessibility

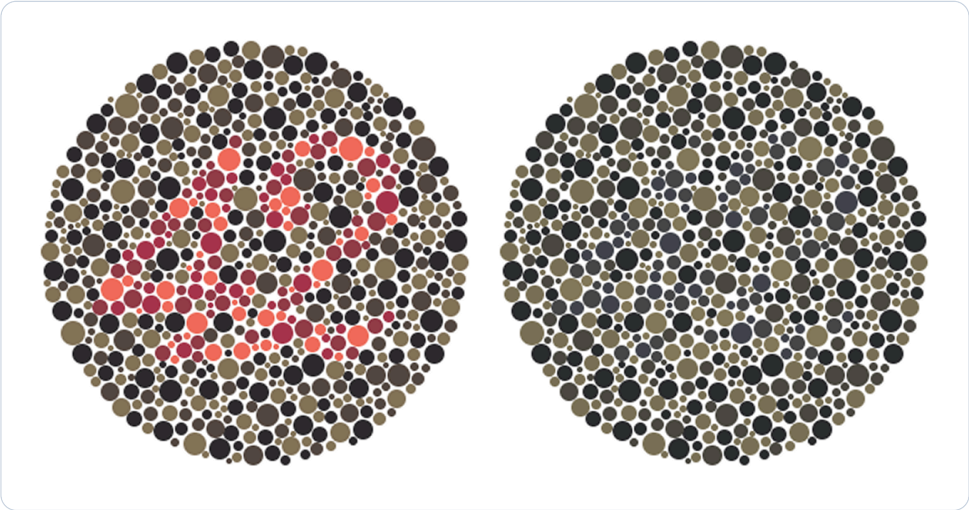

Figure 6: A comparison between a plate configured for `Deuteranopia` on the left and the same plate viewed through a `Deuteranopia` simulation filter on the right.

The implementation of the interactive Ishihara plate is available in the project's repository: https://github.com/Flaverus/P8_sandbox/tree/main/examples/widget

6.4. (Accessible Drag and Drop Suggestion)

The accessible drag and drop solution presented in this chapter as seen in Figure 7 was not a direct part of the project itself. However, it was explored during the project's development period and fits thematically into the broader accessibility focus of this documentation. The underlying problem emerged during a coordination meeting related to the project and was further investigated out of personal curiosity.

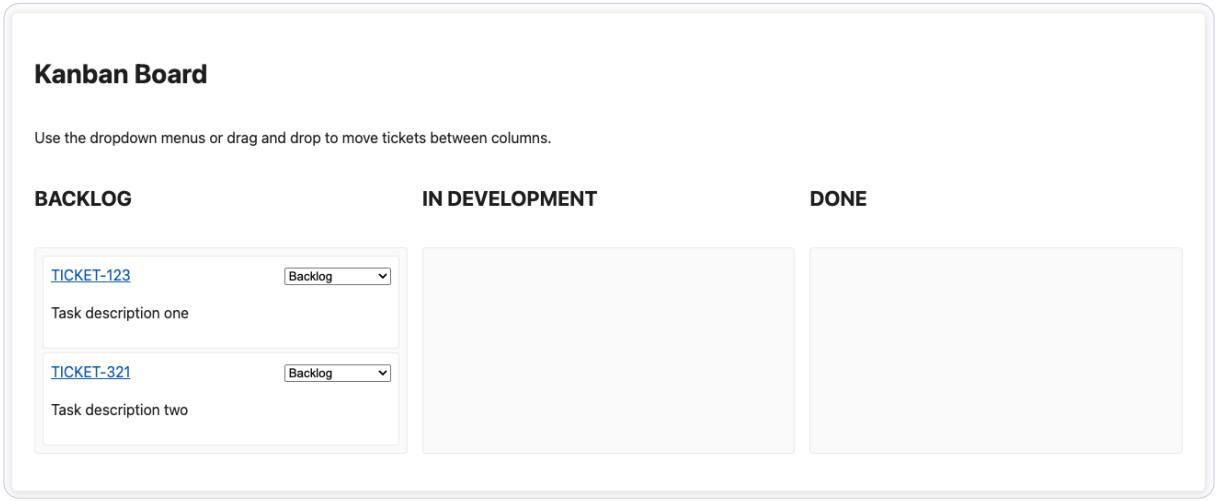

Figure 7: A screenshot of the keyboard accessible drag and drop example.

Because this topic is not part of the project's core implementation, the related theory was intentionally omitted from the main theory chapter. In addition, this section does not analyze the implementation in the same level of detail as the primary project components.

The draggable elements are HTML `<li>` elements with the attribute `draggable="true"` applied to them. Within the context of this example, they represent tickets in a kanban board. Each ticket additionally contains a `<select>` element that is populated with available placement options, allowing the same interaction to be performed using a keyboard.

The tickets can be moved between different states such as *BACKLOG* and *IN DEVELOPMENT*, which are represented by `<ol>` elements. The basic HTML structure is shown in Listing 36.

```
<div class="kanban">
  <section>
    <h3>BACKLOG</h3>
    <ol id="area-one" data-identifier="Backlog">
      <li draggable="true" id="one">
        <article>
          <div class="tile-header">
            <a href="#">TICKET-123</a>
            <label>
              <span class="visually-hidden">Move TICKET-123 to:</span>
              <select></select>
            </label>
          </div>
          <p>Task description one</p>
        </article>
      </li>
    </ol>
  </section>

  <section>
    <h3>IN DEVELOPMENT</h3>
    <ol id="area-two" data-identifier="In Development"></ol>
  </section>
</div>
```

Listing 36: The basic HTML structure with draggable `<li>` elements that can be moved between `<ol>` containers.

To support drag and drop interactions, the elements require additional JavaScript functionality. The `dragstartHandler()`, `dragoverHandler()`, and `dropHandler()` functions provide the core interaction logic.

The `dragstartHandler()` stores the identifier of the dragged element for later use. The `dragoverHandler()` prevents the default browser behavior so dropping remains possible. Finally, `dropHandler()` appends the dragged `<li>` element to the target `<ol>` element and updates the available keyboard selection options, as shown in Listing 37.

```
const dragstartHandler = ev => {
  ev.dataTransfer.setData("text", ev.target.id);
}

const dragoverHandler = ev => {
  ev.preventDefault();
}

const dropHandler = ev => {
  ev.preventDefault();
  const data   = ev.dataTransfer.getData("text");
  const target = ev.target.closest('ol');
  if(target) {
    target.appendChild(document.getElementById(data));
    updateSelectMenus();
  }
}

const initBoardEvents = () => {
  const allOLs = document.querySelectorAll('.kanban ol');
  allOLs.forEach(ol => {
    ol.addEventListener('drop', dropHandler);
    ol.addEventListener('dragover', dragoverHandler);
  });

  const allLIs = document.querySelectorAll('.kanban li');
  allLIs.forEach(li => {
    li.addEventListener('dragstart', dragstartHandler);
  });
};
```

Listing 37: JavaScript event handlers used to support drag and drop interactions between the `<ol>` containers.

The accessibility-related addition that differentiates this example from the implementation shown in Mozilla's HTML Drag and Drop API documentation [37] is the `updateSelectMenus()` function. This function dynamically updates all `<select>` elements with the available target columns, allowing tickets to be moved entirely through keyboard interaction.

The `selectHandler()` function then moves the corresponding `<li>` element to the selected column whenever the value of the `<select>` element changes, as demonstrated in Listing 38.

Although the implementation is intentionally simple and not heavily optimized, it demonstrates that accessible drag and drop interactions can be implemented with relatively little additional complexity.

```
const selectHandler = ev => {
  const targetColumnId = ev.target.value;
  const listItem       = ev.target.closest('li');
  const targetColumn   = document.getElementById(targetColumnId);
  targetColumn.appendChild(listItem);
  updateSelectMenus();
}


const updateSelectMenus = () => {
  const columns = document.querySelectorAll('.kanban ol');
  const selects = document.querySelectorAll('.kanban li select');

  selects.forEach(select => {
    const parentUl        = select.closest('ol');
    const currentColumnId = parentUl.id;
    select.innerHTML      = '';

    columns.forEach(column => {
      const option       = document.createElement('option');
      option.value       = column.id;
      option.textContent = column.getAttribute('data-identifier');

      if (column.id === currentColumnId) {
        option.selected = true;
      }

      select.appendChild(option);
    });

    select.removeEventListener('change', selectHandler);
    select.addEventListener('change', selectHandler);
  });
};
```

Listing 38: Keyboard accessible movement of tickets through dynamically updated `<select>` *elements.*

The implementation of the keyboard accessible drag and drop example is available in the project's repository: https://github.com/Flaverus/P8_sandbox/tree/main/examples/drag-and-drop

7. Discussion

Generally speaking, modern CSS and JavaScript already provide powerful accessibility primitives, and many accessibility concerns do not require large or complex frameworks to be addressed effectively. Native capabilities are often overlooked or underestimated by developers, either due to a lack of awareness or because many modern projects begin directly with frameworks instead of first considering native approaches. In some cases, developers entering the industry are barely exposed to native environments at all, as frameworks are already introduced during apprenticeships before the fundamentals of the web platform itself are fully understood.

Frameworks certainly provide quality-of-life improvements and can simplify development in many scenarios. However, in some areas, avoiding unnecessary abstractions can reduce complexity and improve transparency. Working with native technologies and minimizing dependencies often allows for more direct customization and better adaptation to individual accessibility needs.

It is also remarkable how quickly browser standards evolve. During the timespan of this project, for example, the `contrast-color()` CSS function transitioned from limited browser support to broad support across all major browsers.

At the same time, browser support for CSS and JavaScript features still contains inconsistencies. Certain areas feel incomplete or missing fundamental capabilities. However, these shortcomings should also be viewed in the context of the web platform's continuous development, as existing gaps are gradually closed while new requirements emerge over time.

One example is the extraction of `RGB` values from `oklch()` colors. At the time this project was written, this was not directly possible in a practical way. A reasonable expectation would be for `getComputedStyle()` to return computed `RGB` values even when colors are defined using `oklch()`. Instead, the original `oklch()` value is returned unchanged. To calculate contrast ratios, the implementation therefore had to rely on the HTML `<canvas>` element to extract the required `RGBA` values through `getImageData()`.

Another particularly interesting aspect was working with the experimental `@function` at-rule. Since the feature is still experimental, incomplete behavior is expected to some extent. However, the biggest challenge was the lack of debugging possibilities combined with sparse and partially vague documentation. Developing with this feature often became a trial-and-error process because there was no reliable way to inspect intermediate states during calculations or identify why certain expressions failed. Some supported behaviors are only loosely documented, while other surprisingly useful capabilities are not documented at all despite working reliably in practice.

An additionally challenging aspect is passing relevant intermediate values into calculations. Functions such as `oklch()` allow access to properties like the lightness component of a color, and these values can be manipulated directly within the function itself. However, there is currently no practical way to extract such intermediate values into custom properties for further processing. This makes it difficult to build more complex conditional logic, for example when trying to dynamically alter output values with the `if()` function based on calculated lightness or other color characteristics.

The current binary pass/fail nature of the WCAG 2.x contrast requirements is another aspect worth discussing. The calculations are heavily simplified and even contain known issues, such as the threshold value `0.03928` being used instead of `0.04045`, which originated from a rounding error in the original WCAG draft and later became fixed within the specification. Furthermore, the current model does not properly account for perceptual factors. A contrast ratio may technically satisfy the guideline while still appearing visually uncomfortable or less readable in practice.

This is where APCA introduces promising improvements. The model considers perceptual aspects such as text size, font weight, and surrounding context, acknowledging that readability changes depending on how content is visually presented. Once further refined and standardized, this approach could provide a far more realistic foundation for accessibility contrast evaluation across the web.

8. Conclusions

Improving accessibility and usability for a diverse group of users does not necessarily require large or complex solutions. This project demonstrates that existing web standards can already be extended with relatively little implementation effort to provide a more user-centric and customizable experience for how web content is presented.

The Preferences Widget in particular shows how native browser capabilities and CSS custom properties can be combined to create flexible accessibility configurations on a website-specific level. At the same time, this approach introduces an important challenge. The more configuration options become available, the more visual combinations and states must be considered, implemented, and tested. Combining multiple accessibility preferences quickly results in a large number of possible variations. This complexity represents the biggest limitation of such an approach and makes it difficult to realistically cover every possible combination. In practice, implementations therefore need to focus on the settings that provide the greatest benefit for the broadest group of users.

8.1. Limitations

The scope of this project was limited and therefore no large-scale user testing was conducted to quantitatively validate the assumptions and implementations presented throughout this documentation. In addition, perceived contrast remains partially subjective. While contrast calculations and perceptual models can improve accessibility for many users, no universal solution exists that guarantees optimal readability and visual comfort for everyone.

APCA represents a significant improvement over the current relative luminance approach used in WCAG 2.x because it considers perceptual aspects such as text size, font weight, and surrounding context. However, the underlying theory and calculations are considerably more complex than the current WCAG 2.x model. Furthermore, APCA is still under active development and has not yet been finalized or officially established as a web standard. As a result, it is still too early to draw definitive conclusions regarding its long-term adoption and practical impact.

8.2. Future Work

In its current form, the Preferences Widget cannot yet be integrated directly into the Kolibri Web UI toolkit. Although this was not an official part of the project scope, the widget has undergone user testing to evaluate its usability and effectiveness, and a draft proposal for integration into Kolibri will be built right after this project.

The example application for the custom contrast color function could also be extended with APCA support in the future. This would make it possible to evaluate not only WCAG 2.x contrast requirements but also the future WCAG 3.0 perceptual contrast model once the specification becomes finalized and publicly established.

Another valuable next step would be conducting a survey focused on identifying the most important accessibility customization needs users expect from modern websites. Such research could help determine which configuration options provide the greatest practical value for the initial implementation of the Preferences Widget and which accessibility preferences are most relevant in everyday web usage.

Bibliography

[1] Florian Schnidrig, "Accessibility on the Web – Where We Stand." Accessed: Apr. 13, 2026. [Online]. Available: https://accessible-web-initiative.gitbook.io/accessibility-on-the-web-where-we-stand

[2] Mozilla Corporation, "Using media queries." Accessed: Mar. 09, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Media_queries/Using

[3] Mozilla Corporation, "prefers-reduced-motion." Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-reduced-motion

[4] Mozilla Corporation, "prefers-contrast." Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-contrast

[5] Mozilla Corporation, "forced-colors." Accessed: Mar. 09, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/forced-colors

[6] Mozilla Corporation, "prefers-reduced-transparency CSS media feature." Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-reduced-transparency

[7] Mozilla Corporation, "prefers-color-scheme." Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-color-scheme

[8] Mozilla Corporation, "inverted-colors." Accessed: Mar. 09, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-color-scheme

[9] Mozilla Corporation, "Window: matchMedia() method." Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Window/matchMedia

[10] Naftali Peles, "[mediaqueries] CSS Media Feature for Color Vision Adjustments." Accessed: Mar. 16, 2026. [Online]. Available: https://github.com/w3c/csswg-drafts/issues/10335

[11] World Wide Web Consortium, "Media Queries – disposition of comments." Accessed: Apr. 16, 2026. [Online]. Available: https://www.w3.org/Style/css3-updates/css3-mediaqueries-comments

[12] Mozilla Corporation, "@media." Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media#media_types

[13] Mozilla Corporation, “Using CSS custom properties (variables).” Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Cascading_variables/Using_custom_properties

[14] Mozilla Corporation, “@property CSS at-rule.” Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@property

[15] Mozilla Corporation, “syntax CSS at-rule descriptor.” Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@property/syntax

[16] Mozilla Corporation, “CSS: registerProperty() static method.” Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/CSS/registerProperty_static

[17] Mozilla Corporation, “CSSStyleDeclaration: setProperty() method.” Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/CSSStyleDeclaration/setProperty

[18] Mozilla Corporation, “@container CSS at-rule.” Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/de/docs/Web/CSS/Reference/At-rules/@container

[19] Mozilla Corporation, “Attribute selectors.” Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Selectors/Attribute_selectors

[20] World Wide Web Consortium, “CSS Custom Properties for Cascading Variables Module Level 1.” [Online]. Available: https://www.w3.org/TR/css-variables-1/#syntax

[21] Fynn Ellie Becker, “CSS Custom Property toggles for themes.” Accessed: Apr. 29, 2026. [Online]. Available: https://fynn.be/blog/css-custom-property-toggles-themes/

[22] Mozilla Corporation, “var() CSS function.” Accessed: Apr. 29, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/var

[23] Mozilla Corporation, “contrast-color() CSS function.” Accessed: May 04, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/contrast-color

[24] Mozilla Corporation, “light-dark() CSS function.” Accessed: May 04, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/light-dark

[25] Mozilla Corporation, "@function CSS at-rule." Accessed: May 04, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@function

[26] Mozilla Corporation, "View Transition API." Accessed: May 21, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/View_Transition_API

[27] Brian Smith, "New functions, gradients, and hues in CSS colors (Level 4)." Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/blog/css-color-module-level-4/

[28] World Wide Web Consortium, "Contrast (Minimum) (Level AA)." Accessed: Apr. 20, 2026. [Online]. Available: https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html

[29] World Wide Web Consortium, "Relative luminance." Accessed: Apr. 20, 2026. [Online]. Available: https://www.w3.org/WAI/GL/wiki/Relative_luminance

[30] Andrew Somers, "Why APCA as a New Contrast Method?." Accessed: Apr. 20, 2026. [Online]. Available: https://git.apcacontrast.com/documentation/WhyAPCA.html

[31] Ericka O'Connor, "Orange You Accessible? A Mini Case Study on Color Ratio." Accessed: Apr. 27, 2026. [Online]. Available: https://www.bounteous.com/insights/2019/03/22/orange-you-accessible-mini-case-study-color-ratio/

[32] Mozilla Corporation, "<canvas> HTML graphics canvas element." Accessed: May 11, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/canvas

[33] Mozilla Corporation, "Canvas API." Accessed: May 11, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API

[34] Mozilla Corporation, "ImageData." Accessed: May 11, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/ImageData

[35] Geoffrey Potjewyd, "The Color Code." Accessed: Apr. 27, 2026. [Online]. Available: https://theophthalmologist.com/issues/2022/articles/sep/the-color-code

[36] Karl Grovesr, "Web Accessibility Testing: What Can be Tested and How." Accessed: Mar. 02, 2026. [Online]. Available: https://karlgroves.com/web-accessibility-testing-what-can-be-tested-and-how/

[37] Mozilla Corporation, "HTML Drag and Drop API." Accessed: May 21, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/HTML_Drag_and_Drop_API

List of Figures

Figure 1 This chart illustrates the spatial dependence of human contrast sensitivity using text samples. [30] 21
Figure 2 A high-contrast demonstration plate designed to remain legible across all common vision types. 24
Figure 3 A screenshot of the preferences widget with page-specific settings for reduced motion and color blindness. 27
Figure 4 A screenshot of the configuration interface for the custom contrast color function. 36
Figure 5 A screenshot of the interactive Ishihara plate configured with colors from the Kolibri palette. 38
Figure 6 A comparison between a plate configured for `Deuteranopia` on the left and the same plate viewed through a `Deuteranopia` simulation filter on the right. 39
Figure 7 A screenshot of the keyboard accessible drag and drop example. 40

List of Listings

Listing 1 Syntax of a media query, where `<media-query-list>` contains `<media-type>` values, `<media-feature>` values, and logical operators. . . 3
Listing 2 Example of a media query that changes the background color on smaller screens using the `screen` media type and the `max-width` media feature. . 4
Listing 3 Checking whether the user's system settings prefer a dark color scheme. . 5
Listing 4 Defining a primary color custom property that can be reused throughout the website. 7
Listing 5 Defining a primary color custom property using the `@property` at-rule. . 8
Listing 6 Using `registerProperty()` and `setProperty()` in JavaScript to define CSS custom properties. 8
Listing 7 Reading a CSS custom property in JavaScript using `getPropertyValue()`. 8
Listing 8 Using the CSS `@container` at-rule to disable animations based on a custom property. 9
Listing 9 Using a CSS attribute selector to switch to a dark color theme based on a custom property. 9
Listing 10 Both `' '` and `initial` are valid values for custom properties. 10
Listing 11 Assigning either `' '` or `initial` depending on the active theme creates the basis for the toggle logic. 10
Listing 12 When a custom property becomes invalid, `var()` automatically uses the fallback value. 11
Listing 13 Setting the text color of a button to black or white based on the background color. 12
Listing 14 Defining text and background colors based on the active color theme using custom properties. 12
Listing 15 A custom function that provides behavior similar to `contrast-color()` and was created before that function became widely available. 13
Listing 16 A custom page transition that swipes the previous view out to the left while moving the new view in from the right. 15
Listing 17 Using `rgb()` to define a color from the Kolibri palette with 50% opacity. 16
Listing 18 Using `hsl()` to define a color from the Kolibri palette with 60% opacity. 17
Listing 19 Using `oklab()` and `oklch()` to define a color from the Kolibri palette with 70% opacity. 17
Listing 20 Using `hwb()` to define a color from the Kolibri palette with 30% opacity. 18
Listing 21 Step-by-step calculation of relative luminance based on the W3C sRGB formula. [29] 20
Listing 22 Creating a purple 100 × 100 pixel square and extracting the RGBA values from a 10 × 10 pixel region starting at coordinates x = 10 and y = 10. . 23

Listing 23 The `setAccessibilityProperty()` function stores a value in `localStorage` and updates the corresponding custom property on `:root`. 28
Listing 24 The `getAccessibilityProperty()` function returns a saved value or falls back to the system setting. 29
Listing 25 Initializing the global custom properties with stored values or system defaults. 29
Listing 26 Synchronizing the selected radio buttons with the values stored in the custom properties. 30
Listing 27 Updating the user's preference when the selected radio button changes. .30
Listing 28 Overriding the default custom properties with the CSS attribute selector on `:root` to apply a dark color scheme. 31
Listing 29 Applying a dedicated style when both dark mode and high contrast mode are enabled. 32
Listing 30 Applying additional styles based on custom properties using the `@container` rule and a `style()` query. 32
Listing 31 Having custom properties defined with `@property` to be able to add a transition within a `@container` rule. 33
Listing 32 Enabling a view transition for a property change using the `startViewTransition()` method. 34
Listing 33 Increasing the duration of the view transition to create a smoother visual effect. 34
Listing 34 A custom contrast function that softens pure black and white and optionally mixes in a portion of the original color. 35
Listing 35 Extracting the `RGBA` values of any CSS color using the HTML `<canvas>` element. 37
Listing 36 The basic HTML structure with draggable `<li>` elements that can be moved between `<ol>` containers. 41
Listing 37 JavaScript event handlers used to support drag and drop interactions between the `<ol>` containers. 42
Listing 38 Keyboard accessible movement of tickets through dynamically updated `<select>` elements. 43

List of Aids

Artificial Intelligence and Language Models

Tool: Gemini (Google), Version 3 Flash.

Type of Usage: Used for linguistic refinement of drafts and formating mathematical formulas (Relative Luminance).

Extent: Specific paragraphs regarding WCAG luminance calculations were refined using the tool. All technical facts and code were manually verified for accuracy.

Tool: ChatGPT (OpenAI), Version GPT-5.5.

Type of Usage: Used for linguistic refinement of drafts and improvement of readability and text flow.

Extent: Specific sections of the documentation were refined using the tool to improve wording, transitions, and overall readability. The original content, technical facts, code, and project structure remained author-created and were manually verified for accuracy.

MSE-Declaration of Originality

The project reports (P7 and P8) and the master's thesis (P9) must be accompanied by the following declaration of originality and signed:

I hereby declare that any individual work submitted for assessment is entirely the product of my own effort:

- That I have correctly cited all text passages that do not originate from me, in accordance with standard academic citation rules (e.g., APA or IEEE), and that I have clearly mentioned all sources used;
- That I have declared in footnotes or in an "List of Aids" all aids used (AI assistance systems such as chatbots [e.g., ChatGPT], translation [e.g., DeepL], paraphrasing [e.g., QuillBot]) or programming applications [e.g., GitHub Copilot] and indicated their use at the corresponding text passages;
- That I have acquired all intangible rights to any materials I may have used, such as images or graphics, or that these materials were created by me;
- That the topic, the thesis, or parts of it have not been used in an assessment of another module, unless this has been expressly agreed with the lecturer in advance and is stated as such;
- That I am aware that my work may be checked for plagiarism and for third-party authorship of human or technical origin (artificial intelligence);
- That I am aware that the University of Applied Sciences and Arts Northwestern Switzerland FHNW will pursue a violation of this declaration of authenticity and that disciplinary consequences (reprimand or expulsion from the study program) may result from this.

Windisch, _____

Location / Date

Florian Schnidrig

First Name / Surname

_____

Signature